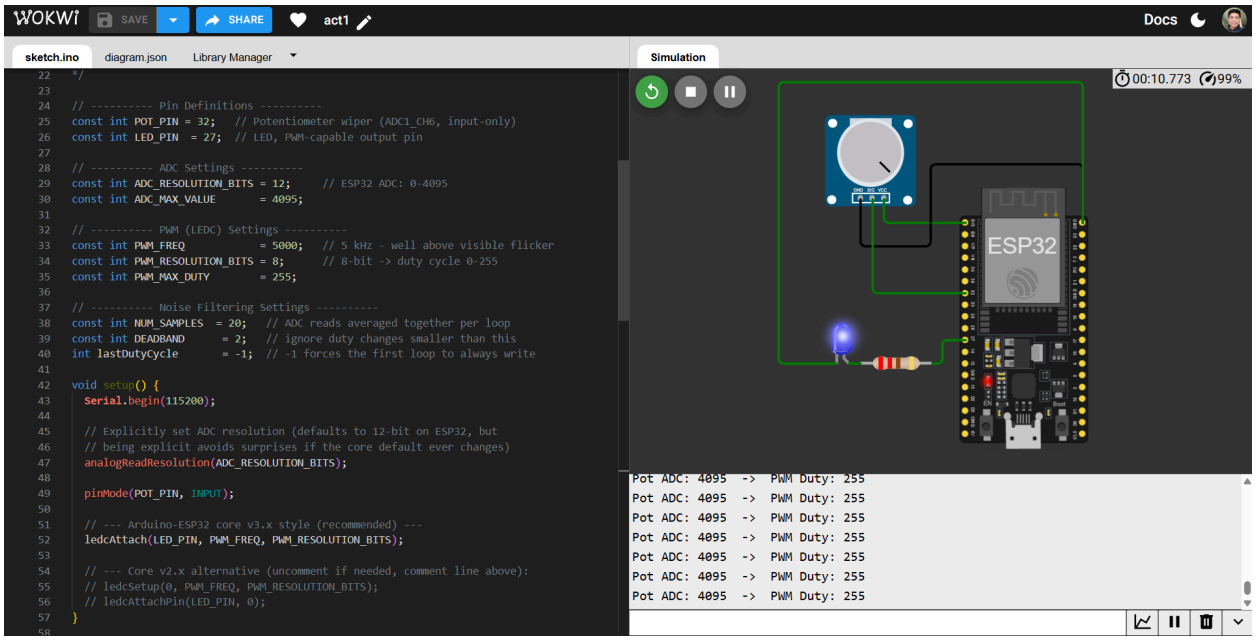
Name: Aaron Ludwig A. Altar **ESP Activity#1**

Section: CPE4B **Date: Sept. 18, 2026**

Potentiometer/LED Control

Wokwi Simulation Screenshot




```
*/

// ---------- Pin Definitions ----------
const int POT_PIN = 32;   // Potentiometer wiper (ADC1_CH6, input-only)
const int LED_PIN  = 27;  // LED, PWM-capable output pin

// ---------- ADC Settings ----------
const int ADC_RESOLUTION_BITS = 12;     // ESP32 ADC: 0-4095
const int ADC_MAX_VALUE       = 4095;

// ---------- PWM (LEDC) Settings ----------
const int PWM_FREQ            = 5000;   // 5 kHz - well above visible flicker
const int PWM_RESOLUTION_BITS = 8;      // 8-bit -> duty cycle 0-255
const int PWM_MAX_DUTY        = 255;

// ---------- Noise Filtering Settings ----------
const int NUM_SAMPLES  = 20;   // ADC reads averaged together per loop
const int DEADBAND     = 2;    // ignore duty changes smaller than this
int lastDutyCycle      = -1;   // -1 forces the first loop to always write

void setup() {
  Serial.begin(115200);

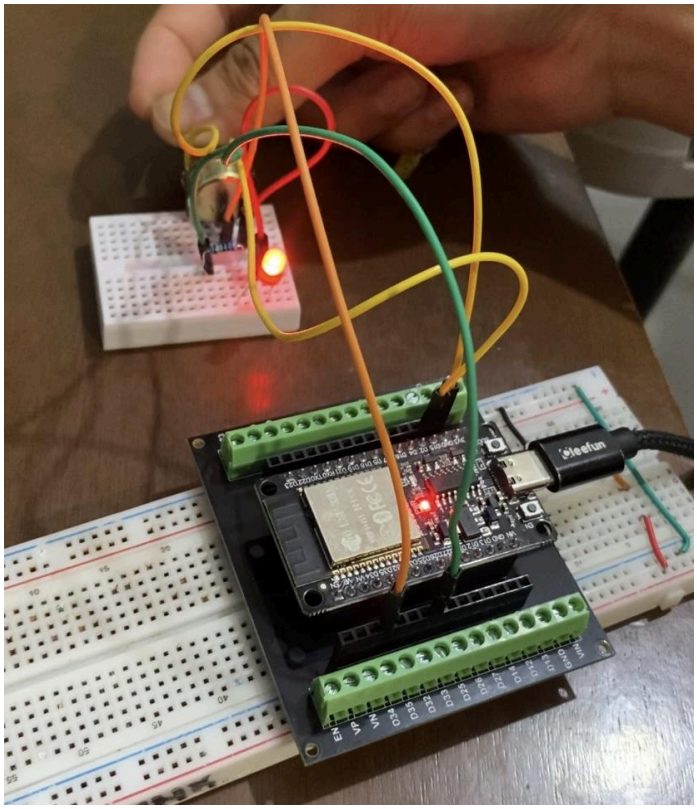
  // Explicitly set ADC resolution (defaults to 12-bit on ESP32, but
  // being explicit avoids surprises if the core default ever changes)
  analogReadResolution(ADC_RESOLUTION_BITS);

  pinMode(POT_PIN, INPUT);

  // --- Arduino-ESP32 core v3.x style (recommended) ---
  ledcAttach(LED_PIN, PWM_FREQ, PWM_RESOLUTION_BITS);

  // --- Core v2.x alternative (uncomment if needed, comment line above):
  // ledcSetup(0, PWM_FREQ, PWM_RESOLUTION_BITS);
  // ledcAttachPin(LED_PIN, 0);
}
```

Actual Experiment



Name: Aaron Ludwig A. Altar **ESP Activity#1**
Section: CPE4B **Date: Sept. 18, 2026**

Code Snippet

```
// ---------- Pin Definitions ----------
const int POT_PIN = 32;   // Potentiometer wiper (ADC1_CH6, input-only)
const int LED_PIN  = 27;  // LED, PWM-capable output pin

// ---------- ADC Settings ----------
const int ADC_RESOLUTION_BITS = 12;     // ESP32 ADC: 0-4095
const int ADC_MAX_VALUE       = 4095;

// ---------- PWM (LEDC) Settings ----------
const int PWM_FREQ            = 5000;   // 5 kHz - well above visible
flicker
const int PWM_RESOLUTION_BITS = 8;      // 8-bit -> duty cycle 0-255
const int PWM_MAX_DUTY        = 255;

// ---------- Noise Filtering Settings ----------
const int NUM_SAMPLES  = 20;   // ADC reads averaged together per loop
const int DEADBAND      = 2;   // ignore duty changes smaller than this
int lastDutyCycle       = -1;  // -1 forces the first loop to always write

void setup() {
  Serial.begin(115200);

  // Explicitly set ADC resolution (defaults to 12-bit on ESP32, but
  // being explicit avoids surprises if the core default ever changes)
  analogReadResolution(ADC_RESOLUTION_BITS);

  pinMode(POT_PIN, INPUT);

  // --- Arduino-ESP32 core v3.x style (recommended) ---
  ledcAttach(LED_PIN, PWM_FREQ, PWM_RESOLUTION_BITS);

  // ledcSetup(0, PWM_FREQ, PWM_RESOLUTION_BITS);
  // ledcAttachPin(LED_PIN, 0);
}

void loop() {
  // --- Oversample the ADC to cancel out electrical noise ---
```

Name: Aaron Ludwig A. Altar **ESP Activity#1**

Section: CPE4B **Date: Sept. 18, 2026**

```
  long total = 0;
  for (int i = 0; i < NUM_SAMPLES; i++) {
    total += analogRead(POT_PIN);
    delayMicroseconds(100); // brief gap between samples
  }
  int potValue = total / NUM_SAMPLES; // averaged 0-4095 value

  // Map the averaged ADC value to a PWM duty cycle (0-255)
  int dutyCycle = map(potValue, 0, ADC_MAX_VALUE, 0, PWM_MAX_DUTY);
  dutyCycle = constrain(dutyCycle, 0, PWM_MAX_DUTY); // safety clamp

  // --- Deadband filtering: only update the LED on a real change ---
  bool changedEnough = abs(dutyCycle - lastDutyCycle) > DEADBAND;
  bool isEndpoint    = (dutyCycle == 0 || dutyCycle == PWM_MAX_DUTY);

  if (changedEnough || isEndpoint) {
    ledcWrite(LED_PIN, dutyCycle);   // v3.x core
    // ledcWrite(0, dutyCycle);      // v2.x core, use instead

    lastDutyCycle = dutyCycle;
  }

  // Debug output
  Serial.print("Pot ADC: ");
  Serial.print(potValue);
  Serial.print("  ->  PWM Duty: ");
  Serial.println(dutyCycle);

  delay(20);
}
```

Video Link:

[Activity#1 - Video Link]